NOTE A classic complaint (with which I sympathize) is the use of exceptions for process flow whenever an error or unexpected status code is returned. Not all errors are exceptional, throwing exceptions is relatively expensive computationally, and it often means that every call made with a client needs custom exception handling. This design sometimes makes code generation seem more like a burden than a benefit.

Another, subtler issue arises when you use code generation with two separate but related OpenAPI documents, such as a products API and a cart API. If you use the techniques in this chapter to generate the clients and then try to follow this simple sequence, you'll run into trouble:

1. Retrieve a `Product` instance from the products API by using `ProductsClient.Get()`.
2. Send the retrieved `Product` to the cart API by using `CartClient.Add(Product)`.

Unfortunately, the generated `Product` type retrieved from the products API is a different type from the generated `Product` type that the `CartClient` requires, so this code won't compile. Even if the type has the same properties and is serialized to the same JSON when it's sent to the client, *C# considers the objects to be different types* and won't let them swap places. You must copy the values manually from the first `Product` instance to a new instance. These complaints are mostly small niggles and paper cuts, but they can add up when you run into them often.

11.6.3 Tooling often lags the specification

Another factor to consider is the many groups that are involved in generating an OpenAPI document and generating a client:

- The Open API specification is a community-driven project written by the OpenAPI Initiative group.